

Examen terminal - 2ème session

Durée 1h30
(Corrections)

Exercice 1 (Exclusion mutuelle distribuée - 6 pts)

Nous souhaitons développer un système de gestion d'exclusion mutuelle distribuée. Le principe, un serveur centralise les demandes des différents clients distants. Lorsqu'un client demande à entrer en section critique, soit le serveur répond par l'affirmative si aucun autre client est en exclusion mutuelle, soit il met en attente la demande. Lorsqu'un client libère l'exclusion mutuelle, le serveur peut l'accorder à un autre client en attente. Pour les communications réseaux, nous utiliserons le mode non connecté.

1°) Pensez-vous que le choix du mode non connecté est judicieux dans le cadre de cette application ? Donnez des arguments pour et contre.

Solution : (0,5 pt) *Il n'y a pas de dialogue complexe entre le serveur et les clients. Aussi, UDP est un bon choix par rapport à sa légèreté. Par contre, des messages peuvent être perdus, nécessitant la mise en place de compte-à-rebours, par exemple.*

2°) Déterminez les différents types de message échangés dans le système. Vous donnez les structures C correspondantes, en précisant le rôle de chaque champ.

Solution : (0,5 pt) *Un client peut envoyer soit une demande, soit un relâchement de l'exclusion mutuelle. Le serveur envoie simplement un message OK. L'adresse IP du client étant récupérée automatiquement. Donc, dans les deux cas, l'utilisation d'un entier suffit.*

3°) Donnez la portion de code permettant au serveur de créer la socket qui lui permettra de recevoir des demandes. Le numéro de port est fixé par la constante `PORT_ECOUTE` qui vaut 1025 par défaut.

Solution : (1 pt) *Voici une solution :*

```
int sockfd;  
struct sockaddr_in adresseServeur;  
  
sockfd = socket(AF_INET, SOCK_DGRAM, IPPROTO_UDP);  
memset(&adresseServeur, 0, sizeof(struct sockaddr_in));  
adresseServeur.sin_family = AF_INET;  
adresseServeur.sin_port = htons(PORT_ECOUTE);  
adresseServeur.sin_addr.s_addr = htonl(INADDR_ANY);  
bind(sockfd, (struct sockaddr*)&adresseServeur, sizeof(struct sockaddr_in))
```

4°) Pour mémoriser les demandes en attente, nous supposons utiliser une file. Quelles informations le serveur doit-il conserver pour permettre l'envoi ultérieur d'une autorisation d'entrée en section critique ?

Solution : (0,5 pt) *Il suffit de mémoriser l'adresse IP + port du client.*

5°) Nous supposons disposer d'une implémentation d'une file suivant votre réponse à la question précédente, avec les méthodes standard (ajout, récupération de la première entrée, suppression de la première entrée, vérification si la file est vide). Expliquez le fonctionnement de la routine principale du coordinateur (le traitement des requêtes des clients et l'envoi des réponses).

Solution : (1 pt) *Lorsque le serveur reçoit une demande d'entrée en SC, si la file est vide, il envoie une réponse positive au client et place dans la file l'adresse du client. Sinon, il place simplement l'adresse du*

client dans la file. Lorsque le serveur reçoit une fin de SC, il supprime la première entrée de la file. Si la file n'est pas vide, il récupère la première entrée et envoie une réponse positive au client.

6°) Donnez la portion de code permettant au serveur de récupérer l'adresse du client.

Solution : (1 pt) Voici une solution :

```
socklen_t taille = sizeof(struct sockaddr_in);
recvfrom(sockfd, &requete, sizeof(int), 0, (struct sockaddr*)&adresseClient, &
    taille);
```

7°) Si le client qui possède l'exclusion mutuelle tombe en panne, les clients attendront indéfiniment l'entrée en section critique. Proposez une solution permettant de gérer cela. Expliquez ce que cela change pour la routine principale du serveur et précisez les appels systèmes mis en cause.

Solution : (1 pt) Lors de l'envoi de l'entrée en SC du serveur, on place une alarme (en utilisant `alarm`). Si à l'expiration de l'alarme, le client n'a pas renvoyé une sortie, le serveur considère que le client est en panne. Il passe au client suivant. Sinon, l'alarme est désactivée. Par contre, la réception de l'alarme interrompra l'appel `msgrcv`. Il faut donc mettre en place un gestionnaire de signal à l'aide de `sigaction`.

8°) Que se passe-t-il si c'est un message qui est perdu ? Proposez une solution.

Solution : (1 pt) Si le message de demande du client est perdu, il reste bloqué en attente d'un message qui ne viendra jamais. Une solution : envoyer périodiquement un message. Le serveur l'ignore si le client est encore dans la file. Avec le message de fin perdu, idem. Solution : le serveur peut envoyer périodiquement des messages au client en SC qui répond qu'il est encore en SC. Enfin, si le message d'acquiescement est perdu, le client attend en vain et les autres clients sont bloqués. Il est possible de coupler avec la solution précédente : avec un accusé de demande de fin, cela veut dire que le client est en SC et donc, il peut y entrer s'il n'y ait pas encore.

Exercice 2 (Variables partagées - 14 pts)

Nous souhaitons réaliser un système de variables partagées entre plusieurs processus à l'aide d'un segment de mémoire partagée. Nous considérons deux types d'application : le coordinateur et des processus quelconques, que nous appellerons les clients. Le coordinateur crée le segment et se met ensuite en attente de requêtes de la part des clients via une file de messages.

Une variable est caractérisée par un nom de maximum 16 caractères, d'une taille en octets et d'une valeur (ces informations seront sauvegardées dans le segment de mémoire partagée). Un client peut demander la création d'une variable partagée qui sera alors créée dans le segment de mémoire partagée par le coordinateur. Le client peut ensuite accéder à cette variable (à sa valeur). De même, un autre client qui souhaite accéder à cette variable partagée peut récupérer sa position (de sa valeur) auprès du coordinateur.

1°) Questions préliminaires (2,5 pts).

a) Donnez le contenu du segment de mémoire partagée en expliquant la taille des différents éléments.

Solution : (0,5 pts) Le segment contient une liste de : nom de 16 caractères, la taille (1 octet) et les données proprement dites (n octets). Éventuellement, il est possible de donner le nombre actuel de variables.

b) Étant donné que la file de messages unique permet de communiquer entre le coordinateur et tous les clients, rappelez comment s'assurer que les messages soient lus par les bons destinataires (coordinateur ou clients).

Solution : (0,5 pts) Il suffit d'utiliser un type de message spécifique pour le coordinateur (par exemple 1) et les clients envoient leur PID dans le message. Le coordinateur répond en spécifiant comme type, le PID du client. Le coordinateur ne lit que les messages de type 1, les clients que les messages de type égal à leur PID.

- c) Y'a-t-il des problèmes de lecture et/ou d'écriture en utilisant une file de messages, sachant qu'il peut y avoir plusieurs clients qui envoient des requêtes simultanément ?

Solution : (0,5 pts) Non, car la lecture/écriture d'un message est atomique dans les files de messages.

- d) Le coordinateur peut-il envoyer aux clients l'adresse mémoire correspondant au contenu d'une variable partagée contenue dans le segment de mémoire partagée ?

Solution : (0,5 pts) Non. Cette adresse ne pourrait pas être interprétée par le client car il s'agit d'une adresse locale au coordinateur.

- e) Quels cas d'erreur doivent être gérés par le coordinateur dans la gestion des messages ?

Solution : (0,5 pt) Création d'une variable dont le nom existe déjà, mémoire partagée pleine.

2°) Implémentation (5,5 pts).

- a) Déterminez les différents types de message qui sont envoyés dans la file. Pour chacun, proposez une structure en C en expliquant le type et l'intérêt de chaque champ. N'oubliez pas de gérer les cas d'erreur.

Solution : (1,5 pt) L'ajout d'une variable :

- Type (long) = 1 (pour le coordinateur)
- PID (pid_t) = PID du client (pour la réponse)
- Nom (char[16]) = nom de la variable
- Taille (size_t) = taille de la variable

Utilisée aussi pour l'accès à une variable (champ taille non utilisé). Pour la réponse :

- Type (long) = PID du client
- Résultat (unsigned char) = 1 pour OK, 0 pour KO (variable existante, mémoire pleine, etc.)
- Position (off_t) = position dans le segment

- b) En sachant que la clé IPC du segment de mémoire partagée correspond à la constante CLE_SEGMENT et que celle de la file de messages à la constante CLE_FILE, donnez la portion de code permettant à un client de récupérer l'adresse mémoire du segment et de permettre l'accès à la file de messages.

Solution : (1 pt) Voici une solution :

```
int shmId, msgId;
void *adresse;

shmId = shmget((key_t)CLE_SEGMENT, 0, 0);
adresse = shmat(shmId, NULL, 0);
msgId = msgget((key_t)CLE_FILE, 0);
```

- c) Donnez la portion de code permettant au client de créer une variable entière (de type int) nommée tmp et de l'initialiser à 0 (en vérifiant le retour du coordinateur).

Solution : (3 pts) Voici une solution :

```
requete_t requete;
requete.type = 1;
requete.PID = getpid();
sprintf(requete.nom, "tmp");
requete.taille = sizeof(int);

msgsnd(msgId, &requete, sizeof(requete_t) - sizeof(long), 0);

reponse_t reponse;
int *tmp;
msgrcv(msgId, &reponse, sizeof(reponse_t) - sizeof(long), (long)getpid(), 0);

if(reponse.resultat == 1) {
    tmp = adresse + reponse.position;
    *tmp = 0;
}
```

3°) Synchronisation (6 pts).

- a) Dans un premier temps, nous supposons que le nombre maximum de variables est fixé (constante `MAX_VAR`). Un client accède à chaque variable en exclusion mutuelle en écriture et/ou en lecture, mais des variables différentes peuvent être accédées simultanément par plusieurs clients. Proposez une solution à l'aide de sémaphores et dites si les structures et codes précédents doivent être modifiés (et comment).

Solution : (1 pt) Il suffit de créer un sémaphore pour chaque variable (donc un tableau de `MAX_VAR` sémaphores), initialisé à 1. Lorsqu'un client désire accéder à une variable, il fait un `P (-1)` sur le sémaphore correspondant (le coordinateur doit indiquer l'indice de la variable partagée lors de la récupération et/ou la création donc la structure `reponse_t` doit être modifiée); si un autre client accède déjà à la variable, il sera bloqué. Une fois l'accès terminé, il fait un `V (+1)`.

- b) La clé du tableau de sémaphores est stockée dans la constante `CLE_SEM`. Donnez la portion de code permettant de créer et d'initialiser le tableau de sémaphores. Si le tableau existe déjà, il est supprimé et recréé.

Solution : (2 pts) Voici une solution :

```
int semid, i;
unsigned short tab[MAX_VAR];

if((semid = semget((key_t)CLE_SEM, MAX_VAR, S_IRUSR | S_IWUSR | IPC_CREAT |
IPC_EXCL)) == -1) {
    semid = semget((key_t)CLE_SEM, 0, 0);
    semctl(semid, 0, IPC_RMID);
    semid = semget((key_t)CLE_SEM, MAX_VAR, S_IRUSR | S_IWUSR | IPC_CREAT |
IPC_EXCL);
}
for(i = 0; i < MAX_VAR; i++) tab[i] = 1;
semctl(semid, 0, SETALL, tab);
```

- c) Nous souhaitons maintenant permettre à des clients différents de pouvoir accéder simultanément en lecture mais en exclusion mutuelle sur l'écriture. La solution consiste à autoriser les clients à accéder en lecture tant qu'aucun client ne demande à accéder en écriture. Bien entendu, un seul client peut accéder en écriture simultanément et seulement lorsque plus aucun client n'accède en lecture. Proposez une solution complète (sémaphores, rôles, opérations).

Solution : (2 pts) L'idée est d'utiliser un sémaphore `SL` (initialisé à 0) pour chaque variable qui est incrémenté à chaque demande d'accès en lecture et un sémaphore `SE` (initialisé à 0) incrémenté à chaque fois qu'un client désire modifier la variable. Un sémaphore `SA` est utilisé pour l'exclusion mutuelle entre écrivain (initialisé à 1)

Lorsqu'un client désire lire, il fait un `[ATT(SE), V(SL)]`. Il lit puis fait `P(SL)`. Lorsqu'un client désire écrire, il fait un `V(SE)` puis un `[ATT(SL), P(SA)]`. Il modifie puis fait `[P(SE), V(SA)]`.

- d) Supposons maintenant que le nombre de variables n'est plus limité. Est-il possible de trouver une solution efficace? Expliquez.

Solution : (1 pt) Il suffira de passer par la file de messages. L'accès sera garanti par le coordinateur.